Station-Level Bike Rental Demand Forecasting

Anton Wilhelmi and Alexander Harant
Introduction to Machine Learning
Group Project – Final Deliverable

***Abstract*—This project predicts the daily bike rental demand for the 20 most used Capital Bikeshare start stations. We frame this as a supervised regression problem with `total_rentals` as the target variable. The main focus is not just on the final model score, but on showing a clear and reproducible process from raw data to the final evaluation. We combine raw trip, station, and weather data into daily records for each station. To prepare the data, we remove variables that could cause data leakage, drop redundant weather features using correlation analysis, apply one-hot encoding for station IDs, and add past rental data (lag features). All models are evaluated using a chronological train-validation-test split to respect the timeline of the data. Our best model is Gradient Boosting with lag features, which achieves a test RMSE of 22.78, a MAE of 16.92, and an R^2 of 0.663. The clearest result of our project is that time factors matter the most: adding lag features improved the test RMSE for every model we evaluated.**

I. Introduction

Bike-sharing systems depend on local and time-dependent demand. If too few bikes are available at a station, users cannot start their trip. If too many bikes accumulate, redistribution costs increase and docking space is wasted. Accurate demand forecasts can help operators plan bike redistribution in advance and improve the overall service quality.

This project explores whether machine learning can produce reliable daily demand forecasts at the station level. We use real trip data from a public bike-sharing system, combined with weather and calendar information. The problem is well-suited for regression because the output is a numeric count and the input features are measurable in advance.

Rather than focusing only on the final model score, we treat the full pipeline as part of the contribution. We show that preprocessing decisions, especially how past demand is represented, have a stronger impact on model quality than the choice of algorithm alone.

This report is structured as follows: Section III describes the dataset and preprocessing steps. Section V explains the modelling pipeline and evaluation strategy. Section VII presents and compares the results of all models. There is also a comparison with another bike-rental project on the same database. The remaining sections cover ethical considerations, limitations, and a critical reflection.

II. Related Work and Comparable Projects

Several studies have used bike-sharing data to compare regression models and explore the effect of feature engineering on prediction quality. We reviewed five related projects to understand which methods are commonly applied and to justify the design choices in our own pipeline.

TABLE I
Related work and consequences for our project design.

Source	Relevance and consequence for our design
Fanaee-T & Gama (2013) [4]	Original study introducing the UCI Bike Sharing Dataset. Confirms weather and calendar variables as the standard feature groups for this regression task.
UCI Bike Sharing Dataset [3]	Regression benchmark using Capital Bikeshare data from 2011–2012. Our project uses a newer dataset (2020–2024) and models demand at station-day level instead of city-wide hourly demand.
Kaggle Bike Sharing Demand [5]	Many solutions rely on time features, weather variables, and nonlinear regression. Supports our pipeline-first approach over raw-table modelling.
Capital Bikeshare open data [2]	Operational data source for station-level demand construction. Justifies our station-day aggregation and focus on real operational station behaviour.
owenpb [6]	Kaggle solution comparing Linear, Lasso, Ridge, KNN, Decision Tree, Random Forest, and XGBoost. Motivates comparing multiple model families rather than reporting only one algorithm.

The reviewed work shows that strong demand forecasts usually depend on both model choice and feature construction. This is why our paper gives more space to pipeline decisions than to a long list of final plots.

III. Dataset Construction and Used Features

The dataset used in this project is based on the Capital Bikeshare system from the Washington D.C. area [1], [2]. It contains historical trip records, station metadata, and daily weather data from May 2020 to August 2024. The raw trip table was aggregated into a station-day format: each row represents exactly one start station on one calendar day. This reduction aligns rental counts with daily weather information, produces a single well-defined target variable, and avoids mixing row granularities. After filtering to ensure identical station-day keys for both the lag and no-lag variants, the final dataset contains 30,840 rows, split chronologically into 21,410 training, 4,705 validation, and 4,725 test rows (70/15/15).

A. Target Variable and Distribution

The task is a regression problem with a single numeric target: `total_rentals`, the total number of bike trips started at a specific station on a given day. Intermediate count columns for member/casual users or bike types were removed from the predictor set because they are direct components of the same total. Keeping them would create a leakage-like shortcut where the model reconstructs demand instead of forecasting it.

The distribution shows a wide range of demand levels. This justifies using both RMSE and MAE as evaluation metrics,

TABLE II
TARGET VARIABLE DISTRIBUTION (30,840 STATION-DAY ROWS).

Statistic	`total_rentals`
Mean	73.77 rentals per station-day
Median	70 rentals
Std. dev.	39.75 rentals
Min / Max	1 / 377 rentals

since MAE alone does not penalize large prediction errors sufficiently.

B. Station Filtering and One-Hot Encoding

We focused on the 20 most popular start stations by total trip count. Rare stations provide fewer repeated observations, which makes station-specific patterns unstable and weakens model evaluation. The trade-off is explicit: our models represent high-demand stations, not the full bikeshare system.

The station identity was originally stored as a numeric ID. Using this directly as a feature would mislead models into treating the ID as an ordered quantity. We therefore applied one-hot encoding, producing 20 binary columns, one per station [7].

C. Feature Groups

To make accurate predictions, the models need to understand seasonal patterns, weather conditions, and recent demand history. Before finalising the feature set, we analysed pairwise correlations (see Figure 1) to remove near-duplicate weather variables such as `feelslike` and `temp`.

TABLE III
INPUT FEATURES USED IN THE FINAL MODELLING DATASET.

Feature group	Variables and description
Station identity	20 one-hot binary columns from `start_station_id`. Captures demand differences between stations without implying a numeric order.
Weather & daylight	`tempmax`, `humidity`, `windspeed`, `precip`, `precipcover`, `cloudcover`, `snow`, `snowdepth`, `visibility`, `sealevelpressure`, `uvindex`, `sunset_minutes` (daylight duration proxy).
Calendar & time	`month`, `weekday` (seasonality and commuting patterns), `year`, `time_idx` (continuous chronological index for long-term trends).
Temporal memory	`total_rentals_lag_1` (yesterday's demand) and `total_rentals_lag_7` (same weekday last week), both station-specific.

IV. FEATURE ENGINEERING AND DATA DECISIONS

The feature engineering follows a clear order: reduce the data, define the target, select and encode features, decide about scaling, and finally add lag features after the first results showed that weather and calendar information alone was not enough.

A. Station Identity and One-Hot Encoding

The first version of our pipeline used `start_station_id` as a plain numeric column. This is wrong because station 31623 is not larger or more important than station 31245. Machine learning models that use distances, coefficients, or penalties would treat these numbers as meaningful quantities. We therefore replaced the ID column with 20 binary one-hot columns, one for each station. This gives every model family a clean and comparable station representation.

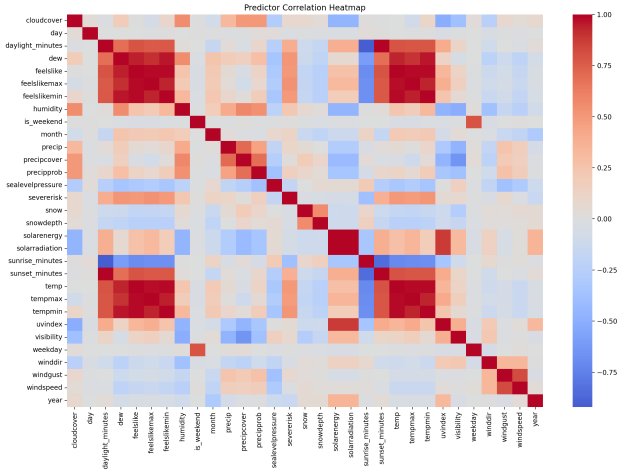

Fig. 1. Correlation matrix used during feature reduction. Strong blocks identified near-duplicate weather variables and supported removing redundant predictors before modelling.

TABLE IV
MAIN PREPROCESSING AND REPRESENTATION DECISIONS.

Decision	Implementation
Station-day aggregation	Group trips by `start_station_id` and date, then merge daily weather data.
Date transformation	Replace raw date strings with `month`, `weekday`, `year`, and `time_idx`.
Leakage control	Remove member/casual and bike-type count columns from input features.
Correlation reduction	Remove near-duplicate variables identified in Fig. 1, such as `solarenergy` and `feelslike`.
No PCA	Keep original features instead of principal components.
One-hot station encoding	Encode `start_station_id` as 20 binary columns [7].
Selective scaling	Standardize inputs only for Linear, Ridge, Lasso, KNN, and Neural Network models.
Lag features	Add `total_rentals_lag_1` and `total_rentals_lag_7` per station.
Fair comparison	Filter both lag and no-lag variants to the same station-day keys.

B. Scaling and PCA

Scaling is applied only where the model needs it. KNN uses distances, regularized linear models use coefficient penalties, and neural networks use gradient-based optimization – all of these are sensitive to the scale of the input values. Decision Tree, Random Forest, and Gradient Boosting split data by thresholds and are not affected by scale. We did not apply PCA because our feature set is already small and every feature has

a clear meaning (temperature, weekday, station, lag demand). Replacing them with abstract components would make the results much harder to explain.

C. Lag Features

Lag features are always computed per station, not across all stations. A global shift would mix demand from different stations and produce incorrect information. The lag value for a given station-day is the same station's rental count from one day earlier (`lag_1`) and from seven days earlier (`lag_7`). Rows without exact lag history are removed. The same rows are also removed from the no-lag dataset so that both variants are evaluated on identical data. This step directly addresses the main weakness of the early models: they used weather and calendar information, but had no memory of recent station demand.

D. Why Representation Came Before More Models

Adding more algorithms does not help if the input data is weak. Our early results showed that the biggest problem was not the choice of algorithm, but the missing temporal memory. We therefore focused on improving the data representation first – station encoding and lag features – before comparing models under one consistent evaluation protocol. This follows a data-centric approach: better inputs lead to better models across the board.

V. Modelling Pipeline and Model Selection

All models share one common pipeline structure. A model comparison is only meaningful when every model receives the same rows, the same target, the same split, and the same evaluation metrics. The repository therefore separates shared utilities from model-specific scripts.

TABLE V
PIPELINE STAGES AND THEIR PURPOSE.

Stage	Purpose
Raw data loading	Read trip, station, and weather files.
Station-day aggregation	Aggregate rentals and merge weather by date.
Feature reduction	Remove leakage columns and redundant predictors.
Lag/no-lag variants	Create two comparable datasets with identical row keys.
One-hot encoding	Encode station identity as 20 binary columns.
Model-specific scaling	Standardize inputs only for models that require it.
Chronological split	70% training, 15% validation, 15% test.
Training and tuning	Fit all models, then optimize hyperparameters on the validation set.
Evaluation	Save metrics, predictions, rankings, and plots.

A. Chronological Split and Scaling

A random split would mix past and future observations, making the task easier than real forecasting. The split is therefore strictly chronological: 70% training, 15% validation, 15% test. The validation set is used for all tuning decisions. The test set is only used once for final reporting.

Scaling is applied only where the model needs it. KNN uses distances, regularized linear models use coefficient penalties, and neural networks use gradient-based optimization – all sensitive to input scale. Decision Tree, Random Forest, and Gradient Boosting split data by thresholds and are not affected by scale. We did not apply PCA because every feature has a clear physical meaning and replacing them with abstract components would make results harder to explain.

B. Model Selection

The nine models were chosen to cover increasing complexity and different learning principles from the course. This is more informative than testing many similar variants of one algorithm.

TABLE VI
MODELS, INPUT VARIANT, SCALING, AND REASON FOR INCLUSION.

Model	Lag	Scale	Reason
Dummy Regressor	no	no	Minimum baseline; all real models must beat it.
Linear Regression	both	yes	Tests whether features contain a roughly linear signal.
Ridge	both	yes	L2 penalty checks whether shrinkage helps.
Lasso	both	yes	L1 penalty adds automatic feature selection.
Decision Tree	both	no	Tests nonlinear threshold rules; prone to overfitting.
KNN	both	yes	Tests whether similar station-days have similar demand.
Random Forest	both	no	Bagging reduces single-tree variance.
Gradient Boosting	both	no	Sequential trees target remaining prediction errors [8].
Neural Network	both	yes	Flexible nonlinear reference to compare against tree ensembles.

VI. Evaluation Strategy

The main ranking metric is RMSE because large demand errors are more problematic in practice than small ones. However, one metric alone is not enough to fully understand model behaviour. The final comparison therefore reports six metrics plus runtime information.

TABLE VII
EVALUATION METRICS AND THEIR INTERPRETATION.

Metric	Interpretation in this project
RMSE	Penalizes large errors more strongly; main ranking metric.
MAE	Average absolute error in rentals; easy to interpret operationally.
Median AE	Typical error on a normal day; less affected by rare extreme demand days.
MAPE	Relative error in percent; useful because stations differ in their average demand level.
R^2	Fraction of variance explained compared to a simple mean baseline.
Explained variance	Checks whether the predicted spread matches the real demand spread.
Fit / predict time	Shows whether accuracy gains come with a computational cost.

A. Hyperparameter Tuning

Hyperparameter tuning was performed in two stages. In the first stage, all models were tuned once using the validation set. This made the model comparison fair because each model family received a suitable basic configuration instead of relying only on default settings.

In the second stage, the strongest candidates from the first comparison were tuned again more carefully. This second tuning focused on parameters that directly control model complexity and generalization, such as the learning rate and number of estimators for Gradient Boosting or the regularization strength for Lasso and the Neural Network. In both stages, the best parameter combination was selected by the lowest validation RMSE. The test set was used only once for the final reported evaluation.

B. Overfitting Diagnosis

Overfitting is diagnosed by comparing train, validation, and test RMSE for each model. If the training error is much lower than the validation and test error, the model has learned patterns that do not generalize well to future data. This is especially visible for KNN and the single Decision Tree. Both models achieve much lower training errors than test errors, which indicates that they fit local training structures too closely.

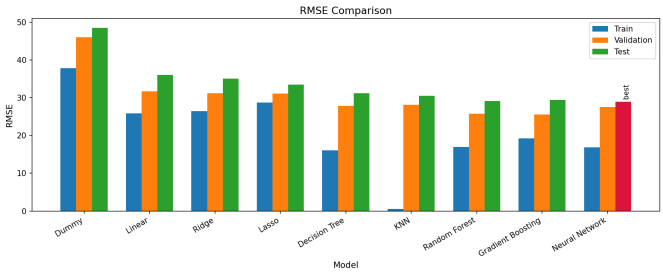

Fig. 2. Train, validation, and test RMSE comparison across models. Large gaps between training and test RMSE indicate overfitting. KNN and the single Decision Tree show the clearest overfitting pattern, while ensemble methods generalize more reliably.

Regularized linear models are more stable, but they may underfit because they cannot capture nonlinear demand patterns sufficiently. Ensemble methods reduce this problem. Random Forest reduces variance by averaging many trees, while Gradient Boosting sequentially corrects remaining prediction errors. The comparison in Fig. 2 therefore supports the final model choice: Gradient Boosting provides a strong balance between flexibility and generalization, but still requires careful tuning because too many or too deep trees could also overfit.

VII. Results

The final comparison contains two variants for each model: without lag and with lag features. This design directly answers the central question of our project: is the performance gain caused by a more complex algorithm, or by better temporal information?

TABLE VIII
FINAL TEST RESULTS AFTER HYPERPARAMETER TUNING.

Model	Lag	MAE	RMSE	R^2	MAPE
Dummy	no	38.14	48.02	-0.499	0.817
Linear	no	27.67	35.75	0.169	0.613
Linear	yes	19.04	24.97	0.595	0.340
Ridge	yes	19.00	24.92	0.596	0.345
Lasso	yes	17.78	23.86	0.630	0.291
Decision Tree	yes	19.47	26.68	0.537	0.282
KNN	yes	19.55	25.37	0.582	0.282
Random Forest	yes	17.17	23.02	0.656	0.295
Gradient Boost	yes	**16.92**	**22.78**	**0.663**	**0.253**
Neural Network	yes	18.89	24.31	0.616	0.299

The three best models are Gradient Boosting (RMSE 22.78, R^2 0.663), Random Forest (RMSE 23.02, R^2 0.656), and Lasso (RMSE 23.86, R^2 0.630), all using lag features. Gradient Boosting achieves the best accuracy but requires the longest training time (9.6 seconds). Random Forest is nearly as accurate and trains much faster. Lasso is the strongest linear model and serves as a transparent benchmark against the ensemble methods.

A. Effect of Lag Features

TABLE IX
EFFECT OF LAG FEATURES ON TEST RMSE (LOWER IS BETTER).

Model	No lag	Lag	Improvement
Linear	35.75	24.97	30.2%
Ridge	34.87	24.92	28.5%
Lasso	32.91	23.86	27.5%
Random Forest	29.02	23.02	20.7%
Gradient Boost	28.24	22.78	19.3%
Decision Tree	30.58	26.68	12.8%
KNN	27.86	25.37	9.0%
Neural Network	26.26	24.31	7.4%

Lag features improve every model without exception. The improvement is largest for linear models (up to 30.2%) because the lag variables add temporal information that weather and calendar features alone cannot capture. Ensemble trees benefit less in relative terms because they can already extract some temporal structure through nonlinear splits, but they still improve by around 20%.

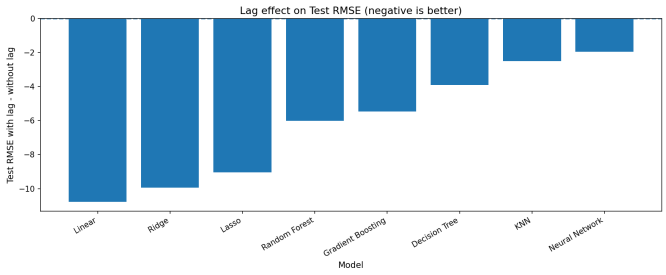

Fig. 3. Test RMSE decreases for every model when station-specific lag features are added. This supports the decision to prioritize temporal representation over adding more algorithms.

B. Hyperparameter Tuning Effect

All nine models were first trained with an initial parameter grid to find a reasonable starting point. For fine-tuning, we selected three models that represent different algorithm families: Gradient Boosting as the best overall model, Lasso as the best linear model, and the Neural Network as a non-tree nonlinear reference. Random Forest was not selected for fine-tuning because it belongs to the same tree ensemble family as Gradient Boosting, and a second tree model would not add a new perspective to the comparison. Table X shows all three tuning stages, starting from the sklearn default parameters.

TABLE X
TUNING STAGES FOR THE TOP THREE MODELS (WITH LAG, TEST RMSE).

Model	Default	Initial	Fine-tuned	R^2
Gradient Boost	23.50	22.79	**22.78**	0.663
Neural Network	24.99	24.31	**24.31**	0.616
Lasso	23.89	23.89	**23.86**	0.630

The table reveals two separate effects. The initial parameter grid already produced the largest gains: Gradient Boosting dropped from 23.50 to 22.79, and the Neural Network from 24.99 to 24.31. Lasso did not change in this step, which confirms that the sklearn default of $\alpha = 1.0$ was already a good starting point. The subsequent fine-tuning step added only marginal improvements for Gradient Boosting (-0.01) and Lasso (-0.03), while the Neural Network did not change at all.

The overall conclusion is that the initial parameter grid captured most of the available tuning gain, and that the largest improvements throughout the project came from better data representation rather than from the tuning process itself.

C. Result Interpretation

Gradient Boosting with lag features is the best final model, reaching a test RMSE of 22.78, MAE of 16.92, and R^2 of 0.663. This means that on average, the model's daily rental prediction for a station deviates by about 23 rentals from the true value on unseen test data. Random Forest is a strong and faster alternative with nearly identical accuracy. Lasso shows that a simple regularized linear model becomes competitive once the dataset contains the right temporal features.

The most important result is not the winning model alone. Lag features improve every model, and better regularization improves tuning results across all three algorithm families. Both findings support the same conclusion: for this dataset, improving the input representation and controlling model complexity matters more than choosing a more sophisticated algorithm.

VIII. DISCUSSION AND CRITICAL REFLECTION

The strongest lesson from this project is that data representation mattered more than adding more algorithms. Before lag features, even flexible models missed a large part of the demand structure. After adding station-specific lags, every model improved. This project is therefore not mainly a story about one winning algorithm – it is a story about converting a raw mobility dataset into a representation that matches the forecasting problem.

A. Why Gradient Boosting Performs Best

Gradient Boosting performs best because it builds trees sequentially, where each new tree focuses on the errors of the previous ones. This is useful for bike demand because effects are not purely additive. Rain, weekday, station identity, and recent demand can interact: a rainy weekday at a commuter station behaves differently from a rainy weekend at a leisure station. A linear model can only capture such interactions if they are manually engineered. Gradient Boosting can approximate many of them automatically through tree splits [8].

However, the result should not be interpreted as Gradient Boosting being universally best. The difference between Gradient Boosting and Random Forest is small, and both are ensemble tree methods. This suggests that the important modelling property is not the exact algorithm name, but the ability to capture nonlinear interactions between station, time, weather, and recent demand.

B. Why the Result Is Not Perfect

A test R^2 of 0.663 is useful but not complete. About one third of the variance remains unexplained. The model does not include public events, strikes, road works, station outages, or short-term weather changes within a day. Daily aggregation also hides commuting peaks and sudden demand shocks. The remaining errors therefore reflect not only model weaknesses, but also missing information in the dataset.

The comparison with related work confirms this interpretation. Sathishkumar and Cho report much higher values, with CUBIST explaining about $R^2 = 0.95$ on Seoul Bike data and about $R^2 = 0.89$ on Capital Bikeshare data [9]. Sathishkumar et al. also report strong results for Gradient Boosting on hourly Seoul demand, with about $R^2 = 0.92$ on the test set [10]. These values are higher than ours, but the tasks are easier in one important way: they predict hourly system-level demand, where local station noise is averaged out and hour of day is available as a very strong predictor. Our project predicts demand at station level, where local fluctuations remain visible.

Absolute error values are also not directly comparable across studies. Choi reports a Random Forest test RMSE of 282.63, MAE of 169.57, and $R^2 = 0.77$ for hourly Seoul demand [11]. These RMSE and MAE values are much larger than ours, but the target scale is also much larger because the model predicts total hourly demand rather than rentals per station-day. The more meaningful comparison is therefore the relative error. Xin reports for UCI-DC one-hour-ahead forecasting an MAE of 51.33, RMSE of 75.88, and MAPE of 26.72% for N-BEATS [12]. Our final MAPE of 25.3% is close to this relative error level. This suggests that our prediction quality is plausible, even though our R^2 is lower than in several hourly system-level studies.

Overall, the comparison shows that our model is not state-of-the-art in terms of explained variance, but it is reasonable

for a stricter station-level setup. The main weakness is not only the algorithm; it is the missing information in the representation. Events, outages, holidays, and hourly demand patterns would likely reduce the remaining unexplained variance more than testing many additional model families.

TABLE XI
MAIN LIMITATIONS AND THEIR PRACTICAL CONSEQUENCE.

Limitation	Consequence
Top-20 station focus	Results cannot be generalized to smaller or less popular stations.
Daily aggregation	Rush-hour peaks and short weather changes are not visible to the model.
Missing event variables	Large demand spikes caused by concerts, sports, or holidays may remain unexplained.
Chronological drift	Mobility patterns can change over time; older training data may not reflect current behaviour.
High-demand days	The model may still underestimate rare extreme demand situations.
Weather-only context	Social, infrastructure, and operational factors are mostly absent from the feature set.

C. Overfitting and Underfitting Diagnosis

The model comparison provides a clear picture of the bias-variance trade-off. Linear models without lag features underfit because they cannot describe the temporal structure of demand. They are stable, but too simple for the nonlinear relation between weather, calendar variables, station identity, and recent demand.

KNN and single decision trees show the opposite risk. They can fit local patterns, but they are more sensitive to noise and specific training samples. This is especially problematic in a chronological forecasting task, because the future period may differ from the past. A model that memorizes local training patterns can therefore look good on training data but lose performance on validation and test data.

Ensemble methods reduce this problem. Random Forest reduces variance by averaging many trees, while Gradient Boosting reduces bias by fitting the remaining prediction errors step by step. The final results match the expected bias-variance behaviour discussed in the course: the best models are flexible enough to capture nonlinear structure, but still regularized enough to generalize to future observations.

D. Ethical and Responsible Use

This task is low-risk compared to personal classification tasks because all data is aggregated and no individual user is predicted. However, operational bias is still possible. A model trained only on popular stations may support decisions that favour already busy areas and ignore underserved neighbourhoods. In a real deployment, forecasts should therefore be monitored by neighbourhood, station type, and demand level.

The model should support human planners, not replace their judgement. This is especially important on unusual days, for example during events, disruptions, or extreme weather. In such cases, the model can provide a baseline forecast, but operational decisions should still include local knowledge and manual correction.

E. Next Steps

The most useful improvements are data-related rather than algorithm-related: noitemsep

- Add hourly modelling to capture commuting peaks and short-term weather effects.
- Include holiday, event, station outage, and service disruption variables to explain demand spikes.
- Analyse residuals by station, season, weekday, and demand level to identify systematic weaknesses.
- Monitor temporal drift, as mobility patterns can change over time and older training data may become less representative.
- Carefully extend the station set so that less popular stations are also represented.
- Test whether rolling averages or longer lag windows improve performance beyond lag 1 and lag 7.

The next modelling step should therefore not be a blind search for more algorithms. A better strategy is to keep the current reproducible pipeline and improve the information available to the models. If hourly data, event indicators, and broader station coverage are added, the remaining error can be analysed more precisely and the model can become more useful for real operational planning.

IX. CONCLUSION

This project shows that temporal representation is the most important factor for accurate bike rental demand forecasting at station level. Adding station-specific lag features improved the test RMSE for every evaluated model, which confirms that recent demand history carries information that weather and calendar features alone cannot provide.

The best final model is Gradient Boosting with lag features, reaching a test RMSE of 22.78, MAE of 16.92, and R^2 of 0.663. The result is not only about which algorithm wins, but about why: the final dataset gives the model station identity, weather context, calendar structure, and recent demand memory. Other key decisions – removing leakage-prone count columns, reducing redundant weather variables, and using a strict chronological split – ensured that the evaluation reflects real forecasting conditions.

Future work should move from daily to hourly forecasting, add event and holiday information, and evaluate model performance across all stations rather than only the 20 most popular ones. These steps would make the model more suitable for real operational planning.

AI TOOL USE

AI tools were used for drafting support, language refinement, source search, and LaTeX structuring. They were also used to help check whether the final report structure follows the project guidelines. The dataset construction, code execution, model outputs, final methodological choices, and interpretation were reviewed and controlled by the authors.

REFERENCES

[1] T. Lo, "Capital bikeshare dataset 2020/05–2024/08," Kaggle. [Online]. Available: https://www.kaggle.com/datasets/taweilo/capital-bikeshare-dataset-202005202408

[2] Capital Bikeshare, "System Data." [Online]. Available: https://capitalbikeshare.com/system-data

[3] H. Fanaee-T, "Bike Sharing," UCI Machine Learning Repository, 2013. [Online]. Available: https://archive.ics.uci.edu/dataset/275/bike+sharing+dataset

[4] H. Fanaee-T and J. Gama, "Event labeling combining ensemble detectors and background knowledge," *Progress in Artificial Intelligence*, vol. 2, no. 2–3, pp. 113–127, 2013. DOI: 10.1007/s13748-013-0040-3

[5] Kaggle, "Bike Sharing Demand." [Online]. Available: https://www.kaggle.com/competitions/bike-sharing-demand

[6] O. Bailey, "Kaggle-Bike-Sharing-Prediction," GitHub. [Online]. Available: https://github.com/owenpb/Kaggle-Bike-Sharing-Prediction

[7] scikit-learn, "OneHotEncoder." [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OneHotEncoder.html

[8] J. H. Friedman, "Greedy Function Approximation: A Gradient Boosting Machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001. DOI: 10.1214/aos/1013203451

[9] V. E. Sathishkumar and Y. Cho, "A rule-based model for Seoul Bike sharing demand prediction using weather data," *European Transport Research Review*, vol. 12, no. 1, 2020. DOI: 10.1186/s12544-019-0388-1

[10] V. E. Sathishkumar, J. Park, and Y. Cho, "Using data mining techniques for bike sharing demand prediction in metropolitan city," *Computer Communications*, vol. 153, pp. 353–366, 2020. DOI: 10.1016/j.comcom.2020.02.007

[11] Y. Choi, "Predicting bike-sharing demand in Seoul, South Korea," Master's thesis, University of Amsterdam, 2021. [Online]. Available: https://scripties.uba.uva.nl/download?fid=c5034481

[12] Q. Xin, "Probabilistic bike-sharing demand forecasting under changing weather and seasonal regimes with transformer-based models," *Findings*, 2026. [Online]. Available: https://findingspress.org/article/157499-probabilistic-bike-sharing-demand-forecasting-under-changing-weather-and-se